CRITICAL OUTAGE // SEV-1

INCIDENT ID: NX-2026-9941 | **FACILITY:** Melbourne Hyperscale Core Exchange | **TARGET:** x86_64 Linux / C++20

PACKET RUSH: MASTER TECHNICAL SPECIFICATION

Canonical Benchmark Contract: Architecture, Matching Logic & 100-Point Scoring Model

1. Operational Crisis & Mission Briefing

Following the commissioning of the high-density **Melbourne Hyperscale Core Exchange**, tier-1 carrier **NexusNet** activated its 400 Gbps multi-region backbone switching fabric. Encountering massive packet floods and policy churn, the legacy sequential engine has collapsed into **severe classification bottlenecks and ingress queue saturation**. Packets are queuing at ingress interfaces, and customer traffic is dropping.

PRIMARY OBJECTIVE: Deploy a next-generation C++20 packet classification engine capable of filtering across **13 distinct header fields** at line-rate speed (target: **25.0 Million packets/sec**), low batch latency (P99 batch latency ≤ 2.5 µs across 64-packet batches), compact memory (≤ 32.0 MiB), and real-time updates (≤ 5.0 µs).

2. The 24-Byte Packet Architecture (13 Dimensions)

A rule matches if and only if **all 13 predicates evaluate to true simultaneously (strict logical conjunction)**:

Field Name	Type & Width	Formal Matching Predicate	Wildcard Condition
`src_ip`	uint32_t (4B)	`(pkt.src_ip & mask) == (rule.src_ip.ip & mask)`	prefix_len == 0
`dst_ip`	uint32_t (4B)	`(pkt.dst_ip & mask) == (rule.dst_ip.ip & mask)`	prefix_len == 0
`src_port`	uint16_t (2B)	`pkt.src_port >= low && pkt.src_port <= high`	[0, 65535]
`dst_port`	uint16_t (2B)	`pkt.dst_port >= low && pkt.dst_port <= high`	[0, 65535]
`packet_len`	uint16_t (2B)	`pkt.packet_len >= low && pkt.packet_len <= high`	[20, 65535]
`region_id`	uint16_t (2B)	`rule.region_id.wildcard || pkt.region_id == rule.region_id.value`	wildcard == true
`vlan_id`	uint16_t (2B)	`rule.vlan_id.wildcard || pkt.vlan_id == rule.vlan_id.value`	wildcard == true
`protocol`	uint8_t (1B)	`rule.protocol.wildcard || pkt.protocol == rule.protocol.value`	wildcard == true
`tcp_flags`	uint8_t (1B)	`(pkt.tcp_flags & mask) == (rule.val & mask)`	mask == 0x00
`ingress_iface`	uint8_t (1B)	`rule.ingress_iface.wildcard || pkt.ingress_iface == rule.ingress_iface.value`	wildcard == true
`dscp`	uint8_t (1B)	`rule.dscp.wildcard || pkt.dscp == rule.dscp.value`	wildcard == true
`icmp_type`	uint8_t (1B)	`rule.icmp_type.wildcard || pkt.icmp_type == rule.icmp_type.value`	wildcard == true (Type 0 valid)
`ttl`	uint8_t (1B)	`pkt.ttl >= low && pkt.ttl <= high`	[0, 255]

• **Packet Field Validity Guarantee**: Harness guarantees all packets are structurally valid (packet_len ∈ [20, 65535], vlan_id ∈ [0, 4095], dscp ∈ [0, 63]). No ingress packet validation required.

• **TCP Flags & ICMP Independence**: All 8 bits supported (0x00–0xFF, including CWR 0x80 and ECE 0x40). icmp_type is evaluated strictly as an ordinary 8-bit field independently of protocol.

• **CIDR Mask Generation**: Input prefix signed int32_t (0 ≤ prefix_len ≤ 32). Bitmask generated via uint32_t unsigned arithmetic: static_cast<uint32_t>(~((uint32_t{1} << (32 - L)) - uint32_t{1})).

3. Priority Resolution & Global Ordering Invariant

• **Numerical Priority**: Lower numerical values indicate strictly higher priority (Priority 1 overrides Priority 10).

• **Deterministic Tie-Breaking & RuleId Reuse**: If two matching rules share numerical priority, the rule with the **lower rule_id** wins. A RuleId may be reused after its previous rule has been successfully deleted.

• **Global Ordering Invariant**: The winning rule must strictly minimize (priority, rule_id) globally across all matching candidates. Return Action::DROP if candidate set is empty; return winning rule action exclusively.

• **Default Action**: If no rule matches a packet, return **Action::DROP**.

4. Benchmark Contract & Execution Model

• **Topology**: Pinned single physical core, fixed clock frequency, no background threads.

• **Throughput vs P99 Batch Latency**: 25.0 Mpps corresponds to 2.56 µs per 64-packet batch. P99 batch latency target of ≤ 2.5 µs enforces tail SLA under the same benchmark run.

• **Memory Budget**: Process-wide peak RSS ≤ 32.0 MiB (accounting for glibc arena allocations and indexes).

• **Dynamic Updates**: Transactional rejection of invalid rules, atomic replacement of existing IDs, interleaved execution in Scenario 05 (Target: ≤ 5.0 µs).

5. Multi-Dimensional 100-Point Scoring Model

Metric Dimension	Max Points	Production Target	Mathematical Scoring Formula
`Throughput (T)`	40 pts	25.0 Mpps	`S_T = 40 × min(1.0, T / 25.0)`
`P99 Batch Latency (L)`	25 pts	2.5 µs (64 pkts)	`S_L = 25 × min(1.0, 2.5 / L)`
`Peak Memory RSS (M)`	20 pts	32.0 MiB	`S_M = 20 × min(1.0, 32.0 / M)`
`Dynamic Updates (U)`	15 pts	5.0 µs	`S_U = 15 × min(1.0, 5.0 / U)`

6. Evaluation Benchmark Scenarios

Scenario	Weight	Scale	Evaluation Objective
`01_correctness_small`	15%	1k rules / 5k pkts	Boundary checks, /0 default masks, exact match edge-cases
`02_scalability_medium`	20%	15k rules / 20k pkts	Cache hierarchy bounds, testing sublinear lookup scaling
`03_performance_large`	25%	40k rules / 30k pkts	Vectorization & SIMD throughput stress testing
`04_adversarial_skew`	20%	20k rules / 20k pkts	Dense wildcards (65%+), nested CIDR, overlapping ports
`05_dynamic_heavy`	20%	15k rules + 10k ops	10,000 interleaved dynamic operations (5k inserts / 5k deletes) measuring update latency

7. Programming Interface & Regulations

• Implement pure virtual class IPacketClassifier in starter/classifier.h and starter/classifier.cpp.

• Zero-Tolerance Correctness: Mismatches against golden baseline result in 0.0 points for that scenario.

• Hardware Environment: Bare-metal isolated x86_64 CPU core with fixed clock frequency.